AI Code Review Agent

Step 1 – Planned System Description

1. Introduction

The planned project is an AI-assisted software system called **AI Code Review Agent**, implemented in Python. The main purpose of the system is to analyse Python source code files and automatically generate structured feedback related to code quality, readability, maintainability, and software complexity.

The system is designed to help users identify syntax problems, coding style issues, overly complex functions, and possible improvements in their programs. Instead of manually reviewing source code, the user will provide a Python file, and the system will automatically analyse it using several integrated tools.

This project demonstrates how an intelligent software agent can coordinate external tools and combine their outputs into a meaningful final report. In addition to functionality, the project also focuses on modular software structure, maintainability, testing, documentation quality, and deployment preparation.

2. Planned System and Goal

The planned system will work as a Python-based intelligent assistant for source code review. The user will provide a Python source code file through a command-line interface. After receiving the file, the intelligent agent will coordinate multiple analysis tools and generate a final review report.

The main goals of the system are:

- to automate the code review process,
- to improve code readability and maintainability,
- to identify syntax and style problems,
- to analyse software complexity,
- to provide structured improvement suggestions.

The final output of the system will include syntax validation results, code quality feedback, complexity analysis, and recommendations for improving the source code.

3. AI / Agent-Based Approach

The project uses a single intelligent agent architecture. The agent acts as the central controller of the system and manages communication between the user and the integrated tools.

The agent is responsible for:

1. receiving the Python source code file,

2. selecting the required analysis tools,
3. executing the analysis process,
4. collecting outputs from all tools,
5. processing and combining the collected data,
6. generating the final structured review report.

The agent itself does not directly perform all analysis tasks. Instead, it coordinates several external tools that specialize in different types of analysis. This demonstrates practical AI-assisted workflow automation and modular system design.

The agent-based design also improves flexibility because new tools can later be added without changing the overall system architecture.

4. Planned Tools

Several external tools and modules are planned for the implementation of the system.

Syntax Analysis Tool

The syntax analysis component will use Python's built-in `ast` module. This tool will validate Python source code structure and detect syntax errors before deeper analysis begins.

Style Checking Tool

The style checking component will integrate Pylint. This tool will analyse:

- readability,
- formatting,
- variable naming conventions,
- maintainability-related issues.

The goal of this tool is to improve code quality according to common Python programming standards.

Complexity Analysis Tool

The complexity analysis component will integrate Radon. This tool will measure cyclomatic complexity and identify functions that may be difficult to maintain or understand.

The collected complexity metrics will support refactoring recommendations and maintainability evaluation.

File Reader Module

The project will also include a file reader module responsible for loading Python source code files into the analysis workflow.

Report Generation Module

A report generation module will combine outputs from all tools and transform them into a readable structured report for the user.

5. Planned Programming Concepts

The implementation of the project is expected to use several important software engineering and programming concepts.

Object-Oriented Programming

Object-oriented programming will be used to separate the responsibilities of:

- the intelligent agent,
- analysis tools,
- report generation modules.

This improves maintainability and code organization.

Modular Software Architecture

The system will be divided into independent modules such as:

- tool integrations,
- workflow logic,
- testing modules,
- utility functions.

This modular structure improves readability, scalability, and future extensibility.

Exception Handling

Exception handling using try-except blocks will be implemented to safely manage:

- invalid file paths,
- syntax errors,
- tool execution failures.

This prevents unexpected system crashes during execution.

File Handling

Python file processing mechanisms will be used to:

- load source files,
- validate file contents,
- transfer data into the analysis workflow.

External Library Integration

The project depends on external tools such as Pylint and Radon. These libraries will be integrated through dedicated wrapper modules.

Testing

Testing modules will be created to verify:

- tool functionality,
- workflow correctness,
- input validation,
- error handling.

Version Control

The project will use Git and GitHub for:

- version tracking,
- commit history,
- project organization,
- collaborative development support.

6. Preliminary Project Structure

The planned project structure is designed to separate responsibilities between different components of the system. Independent directories will be used for:

- agent logic,
- analysis tools,
- testing modules,
- reports,
- sample source code files.

Core files such as `main.py`, `requirements.txt`, and `README.md` will remain in the root directory to simplify installation and execution.

This modular structure improves maintainability, readability, testing, and future expansion of the project.